

SCENARIOS

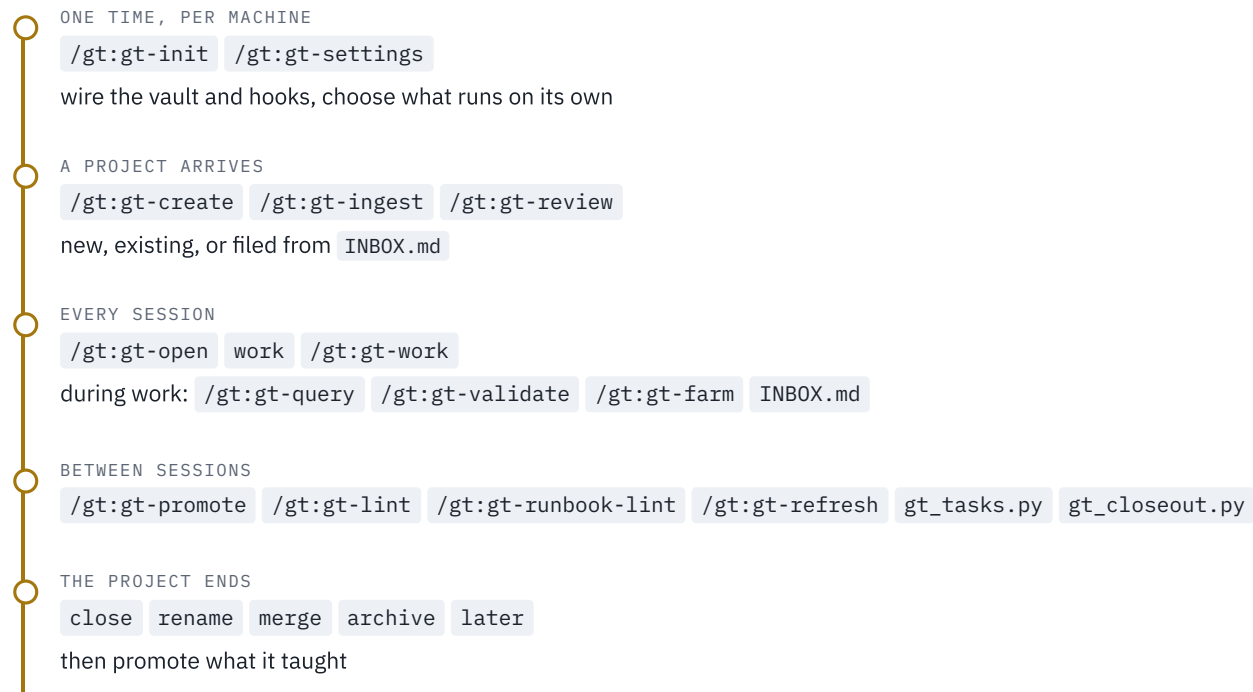
The shape every scenario follows 0 · First day on a new machine 1 · Greenfield, idea to close 2 · Brownfield ingest 3 · Production operations
4 · Farming, sources, refresh 5 · Inbox, review, what's next 6 · The other endings 7 · Maintenance day 8 · The gt-wiki mode Command reference
Files and write direction

Golden Thread Scenarios

Every command in the `gt` and `gt-wiki` plugins, shown inside a complete session path: opening a project, everything that happens while it is open, and every way it can end. Each step names what you say, what the skill reads and writes, and what that buys you.

Every file excerpt is illustrative. Project slugs, hosts, file names and figures are invented stand-ins; the contents shown are examples of shape, not records of fact. Skill names are as installed: `/gt:gt-open`, not `/gt-open`.

The shape that every scenario follows



Two things run without being asked and appear in every scenario:

Session start

Hooks print plugin version, component drift, orphaned workers and unpushed commits. The assistant's first line restates them. Silence is the failure case, so a clean check is announced too.

Every turn

The `UserPromptSubmit` hook injects the six Core rules; the `Stop` hook validates the reply (timestamp, no claimed-file writes). You never invoke these.

First day on a new machine

Situation. You have cloned the vault onto a laptop. The rules are there as files. Nothing is enforcing them.

Step 1: /gt:gt-init

YOU SAY

set up golden thread here

What it does:

- 1. Reads `~/ .claude/vault-config.json` . It is missing, so this is a fresh install.
- 2. Asks two questions: do you already have a vault (yes, give the path) and your domain name, used to customise the schema.
- 3. Asks whether to wire a project right now. You say no.
- 4. Runs `vault_init.py fresh --vault <path> --domain "<domain>"` . Because the vault already exists, almost every line comes back *Already exists*. It creates only what is missing and never overwrites.
- 5. Installs `Projects/golden-thread/core-rules/` if absent and **wires the two hooks** into user-global `~/ .claude/settings.json` : `UserPromptSubmit` for the reminder tier, `Stop` for the validated tier.
- 6. Adds a Golden Thread section to `~/ .claude/CLAUDE.md` .

Verification it tells you to run:

`echo '{}' | ~/ .claude/golden-thread/hooks/inject_core_rules.sh`

The six Core rules print. If they do not, the tier is inert and the skill says so.

WHAT THIS BOUGHT YOU

Without it, the vault's Core rules are documents. One recorded incident had the timestamp rule sitting in memory while silently not applied for dozens of turns. After `gt-init` , it is re-asserted every turn, on every project, and a missing timestamp is a visible alarm that enforcement has broken.

Step 2: /gt:gt-settings

YOU SAY

what does golden thread do on its own?

- 1. Runs `gt_settings.py show` . Prints each setting, its value, whether that is explicit or default, and the options:

SETTING	VALUES	DEFAULT
<code>component_updates</code>	<code>off · report · confirm · auto</code>	<code>report</code>
<code>report_card</code>	<code>off · minimal · full</code>	<code>minimal</code>

- 2. You ask what `auto` means. It runs `gt_settings.py explain component_updates` and reads back the incident that set the default: in one recorded case two of three enforcement scripts existed on one machine only. `auto` never overwrites an installed file

that is newer than the source, because the real drift ran that direction.

3. You say *set report_card to full*. It runs `gt_settings.py set report_card full` and reports the change.

WHAT THIS BOUGHT YOU

Every automatic behaviour is in one registry and every one can be turned off. You are never surprised by the tooling acting on its own.

Step 3: `/gt:gt-lint`

YOU SAY

lint the vault

Runs `gt_lint.py <vault> --queue <vault>/review-queue.md` and walks each finding with a yes/no. On a fresh machine the ones that matter are the three Core checks:

FINDING	MEANING
<code>core-misplaced</code>	a <code>level: core</code> file outside <code>core-rules/</code>
<code>core-no-enforcement</code>	<code>level: core</code> with no <code>enforcement</code> declared
<code>core-unenforced</code>	the declared mechanism is not wired in <code>settings.json</code>

`core-unenforced` is the one that confirms Step 1 worked. It must not fire. The skill refuses to suppress it: suppressing that check re-creates the original bug with a paper trail saying it was fine.

End state. Vault wired, hooks live, settings known, lint clean. You have not opened a project yet.

SCENARIO 1

Greenfield: an idea becomes a closed project

Situation. You want to build a tool that reads a mailbox and triages incoming mail. Nothing exists yet. This scenario shows the whole arc from today.

Session A: capture

/gt:gt-create

YOU SAY

new project called mail-triage. It connects to the mail provider, pulls new mail, classifies it, and feeds the other projects. Personal domain. Runs on my laptop for now.

1. **Gathers inputs, inferring what it can.** Slug `mail-triage`, title "Mail Triage", domain `personal` (from `CONVENTIONS.md`), tags from the taxonomy, topology `local` (you said laptop), no runbook yet, no parent.
2. **Checks for conflict.** `Projects/mail-triage/` must not exist. If it did, it stops and asks: work in the existing folder or choose another name?
3. **Runs the script.** Scaffolding is never done by hand:

```
python3 <plugin>/scripts/vault_init.py create-project \  
  --vault "<vault>" --name mail-triage --title "Mail Triage" \  
  --tags "personal,automation" --domain personal --topology local
```

Shows CREATED lines for `README.md`, `idea.md`, `source.md`, `research.md`, `decisions.md`, `design.md`, `memory/MEMORY.md`, and registers the project in `Projects/README.md`.

4. **Fills `idea.md`** with everything you said, near-verbatim. Not a summary. This file is immutable after creation; it is the origin story you will read in six months to remember why.
5. **Cross-links** to related projects if you named any.

Then it tells you: *Next: run `/gt:gt-open mail-triage` at the start of future sessions.*

WHAT THIS BOUGHT YOU

Every project starts with an identical layout, so every later command knows where to look. The brain dump is captured while it is fresh, and the project is already visible in the vault's Dataview views because `domain` and `stage` are set in frontmatter.

Session B: research

/gt:gt-open mail-triage

YOU SAY

open mail-triage

1. Reads `CONVENTIONS.md` and `PROTOCOL.md` once per session.
2. Reads `README.md` frontmatter first: `stage: idea`, `topology: local`, `domain: personal`. Then the status board and `## Tasks`.

3. Reads `source.md` **before anything else that could lead to touching code**. For a local project this is one line. For a bastion project it is the difference between editing the right host and the wrong one.
4. Reads `idea.md` , `research.md` , `decisions.md` , `design.md` . All nearly empty.
5. Reads `memory/MEMORY.md` , the index only. Does not follow the links.
6. Checks `review-queue.md` and mentions the count once.
7. Summarises: stage, topology, next action, blockers, and how many memory files exist but are not loaded.
8. Asks: *Where do you want to pick up?*

YOU SAY

figure out how to authenticate to the mail provider

Work, with `/gt:gt-query` in the middle

While working, you wonder whether the vault already knows anything about OAuth refresh tokens.

YOU SAY

does the vault have anything on OAuth refresh tokens?

1. Reads `index.md` , scans for a match.
2. Found: follows the wikilink into `Knowledge/` , reads the whole page, follows two or three hops of related links, summarises.
3. If a page is `status: stale` , it says so.
4. Not found in the index: greps `Knowledge/` and `global-memory/` .
5. Still nothing: checks the current project's `memory/MEMORY.md` .
6. Still nothing: says *The vault doesn't have anything on OAuth refresh tokens yet* and offers two options, ingest existing notes with `/gt:gt-ingest` , or write what it knows now as a `status: seed` Knowledge page via `/gt:gt-promote` .
7. Logs the query to `log.md` either way. Repeated queries are the signal that something deserves a page.

WHAT THIS BOUGHT YOU

You never re-derive something the vault already holds, and you never confuse "the vault has nothing" with "I did not look." Say the vault had a page from the `doorbell-camera/device-auth` sub-project on the doorbell vendor's OAuth. The query surfaces it in one hop, and the lesson it recorded about refresh-token expiry is now in this session instead of being relearned.

Capturing a stray thought: `INBOX.md`

Mid-research you think: *the doorbell project should also use the same token refresh pattern*. That belongs to `doorbell-camera` , not here.

YOU SAY

inbox that

The assistant appends one line to `INBOX.md` at the vault root:

```
- [ ] device-auth should share mail-triage's token refresh pattern [project:: doorbell-camera]
```

No priority, no date, no interruption. It shows at the top of `TASKS.md` under **Inbox** so it cannot be lost, and nowhere else so it does not rank.

WHAT THIS BOUGHT YOU

The thought is out of your head and out of the wrong project's `research.md`, in three seconds, without leaving the task.

`/gt:gt-work` at the end of the session

YOU SAY

write back

1. Confirms the project (obvious from context; it does not ask).
2. **Classifies each finding** before writing anything:

FINDING FROM THIS SESSION	SCOPE	DESTINATION
The mail API needs a Desktop OAuth client, not Web	This project	<code>research.md</code>
Refresh tokens for unverified apps expire in 7 days	Cross-project	candidate for <code>Knowledge/</code>
Chose IMAP-over-OAuth rather than the REST API	This project, stable	<code>decisions.md</code> <code>ADR-1</code>
"Remember to try the batch endpoint"	Session-only	not written

3. Appends to `research.md` as dated entries. Append-only; nothing above is edited.
4. Appends `ADR-1` to `decisions.md` with Decision, Context, Rejected alternatives. Only because the decision is stable. A decision that might flip next session stays in `research.md`.
5. Rewrites the changed sections of `design.md` in place. It always describes NOW.
6. Updates `README.md` frontmatter: `stage: idea` becomes `stage: research`. The `## Stage` heading in the body is made to agree, because the property is what the Dataview views read.
7. Adds a task under `## Tasks`:

```
- [ ] Verify the 7-day refresh-token expiry with a real unverified client [p:: 2] [waiting:: agent] [since:: 2026-09-05]
```

8. **Flags the promotion candidate.** *The 7-day expiry looks like it applies beyond mail-triage. Add it to Knowledge now, or flag for `/gt:gt-promote` later?* You say later. It stays in `research.md` with a `→ promote` note. A fact earns its way up by recurring; one project is not enough.
9. Appends to `log.md`:

```
2026-09-05 [work] mail-triage – wrote 2 finding(s), 1 ADR(s), updated design
```

10. Runs the closeout probe. Nothing fires; the project just started.

WHAT THIS BOUGHT YOU

Next session starts from `research.md` and `ADR-1` instead of from your memory of what you tried. The decision you made has its rejected alternatives written down, so nobody proposes the REST API again in three weeks without seeing why it lost.

Session C: a number that matters

The next session opens the same way. You are measuring how fast the parser can classify a backlog, and you reach a claim: *the classifier handles 400 messages a minute on the laptop*. You want that number in `design.md`, which sizes the scheduling.

/gt:gt-validate

YOU SAY

validate that before it goes in design.md

1. **Reduces it to a single falsifiable assertion.** "The classifier processes at least 400 messages per minute on this machine against the seeded test mailbox." Bundled claims return bundled verdicts, which hide which part failed.
2. **Loads the rule pack.** Reads `Projects/mail-triage/validation-rules.md` if it exists. Sub-projects inherit their parent's pack. The pack holds standing invariants the validator enforces whether or not you asked for them, because a requester who has made a domain error will not think to ask about it.
3. **Chooses the validator class.** A number could be wrong: `empirical`. The measurement position could also be blind to something (was the cache warm?): `vantage` as well. Two validators, not one generic reviewer.
4. **Builds the packet.** Exactly three fields:

```
claim:    The classifier processes ≥400 messages/min against the seeded mailbox.
rules:    <pack, verbatim, provenance blocks stripped> + "cold start, cache cleared"
artifact: bench/run_classify.py, mailbox fixture at bench/fixtures/seed_1000.mbox
```

Then **strips** its own numbers, its confidence, the transcript, and every "confirm that" (rewritten as "determine whether"). The validator must never see the reasoning, or it grades the argument and inherits the same blind spot.

5. **Dispatches** one background subagent per class with the matching prompt.
6. **Compares in the right order:** reads the validator's derivation before re-reading its own.
7. Reports one of three verdicts and never rounds `cannot-verify` up to a pass:
 - `confirmed` : re-derived, matches.
 - `refuted` : re-derived, does not match, with the divergence quantified.
 - `cannot-verify` : inputs insufficient, naming what was missing.
8. **Records** the verdict in `research.md` with date, claim, class, outcome. Logs `work`.

Say the `vantage` validator comes back: the fixture is 1,000 messages but 700 are duplicates of 30 senders, so the rate reflects a warm sender cache. Verdict: `refuted`, real cold rate 140/min.

WHAT THIS BOUGHT YOU

The wrong number never reached `design.md`. Per `PROTOCOL.md`, `decisions.md` and `design.md` require `independently verified`; `research.md` accepts `unverified` because it is dated findings, not conclusions. In one recorded incident an unverified analysis reached `decisions.md` as an ADR recommending a four-part production change, and a validator later refuted three of the four parts. The ladder carried the error up because nothing asked how the number was checked. This rung is that question.

The label goes on the figure wherever it appears: `140/min, independently verified`.

Session D: design settles, spec is written

Several sessions later, `design.md` has no open questions. `/gt:gt-work` notices and asks: *Is the design settled enough to write a spec?* You say yes. It creates `spec.md`:

```
# Mail Triage Implementation Spec

## What to change
## Expected behavior
## Tests to write
## Acceptance criteria
- [ ] Cold-start classification ≥100 msg/min on the seeded mailbox
- [ ] Token refresh survives a 7-day unverified-app expiry without user action
```

The spec must be implementable by someone reading only the spec. It references `research.md` and `decisions.md` for the why rather than duplicating them. The stage moves to `design`, then `active` once implementation begins.

WHAT THIS BOUGHT YOU

A handoff artifact. You can give this to a fresh session, another agent, or a person, with zero prior context, and the acceptance criteria are the definition of done.

Session E: the last task, and the close

You check off the last acceptance criterion. `/gt:gt-work` does its normal write-back and then hits the rule: *you just checked off the last open task at p:: 2 or better*. It runs:

```
python3 Projects/golden-thread/tools/gt_closeout.py signals mail-triage
```

Rule `R4` fires (nothing open). It records that it is asking:

```
python3 Projects/golden-thread/tools/gt_closeout.py ask mail-triage gt-work
```

and asks: *mail-triage looks finished: nothing open, all acceptance criteria checked. Close it? yes / no / later, and why?*

YOU SAY

yes, it's shipped and running on the laptop cron.

```
python3 Projects/golden-thread/tools/gt_closeout.py answer mail-triage yes "shipped, on laptop cron"
```

Closing means:

- `README.md` frontmatter `stage: complete`, `pp: 3`.
- Any leftover open task moved to `[p:: 7]`, so it stays in the README as a record but leaves the rollup and stops escalating the project. **Never deleted.**
- A final `research.md` entry saying what shipped.
- Then the promotion check, because a finished project is where the vault's most general lessons usually are.

WHAT THIS BOUGHT YOU

Delivery is not closure; closure is a decision, and the system asks rather than assumes. Every ask and answer is appended to `closeout-signals.jsonl`, so after ten or so answers the thresholds can be tuned to how you actually close things.

After the close: `/gt:gt-promote`

YOU SAY

promote the token expiry finding

1. **Identifies** the item. You named it; it also accepts "review promotion candidates", which scans `research.md` and `decisions.md` for `→ promote` notes.
2. **Determines the destination** with two questions: does this apply beyond `mail-triage`? Yes, `device-auth` hit the same expiry. Does every session need it regardless of project? No.
3. **Writes `Knowledge/OAuth Refresh Token Expiry.md`** with the full frontmatter (`category: reference`, `status: seed`, `created`, `updated`, `sources: []`), adds one line to `index.md`, cross-links from related pages in both directions.
4. Logs with the closed verb vocabulary:

```
2026-09-05 [graduate] mail-triage/research.md → Knowledge/OAuth Refresh Token Expiry.md: 7-day expiry for
unverified apps
```

Had you said it belongs in every session, it would have asked the global-scope question first: zero project slugs, zero project-specific URLs, proven in two unrelated projects, under 30 lines. Failing any of those, it keeps the page in `Knowledge/` and suppresses the promotion.

WHAT THIS BOUGHT YOU

The next project that touches the same OAuth flow finds this in one `/gt:gt-query` hop. The fact moved up the ladder because it recurred, with a human approving the move, and the log records where it came from.

SCENARIO 2

Brownfield: an existing codebase with scattered notes

Situation. You have a repo with a `.claude/memory/` folder, a `CLAUDE.md` full of rules, deploy scripts naming three servers, and a README. None of it is in the vault.

/gt:gt-ingest

YOU SAY
`ingest ~/code/invoice-archiver into the vault as invoice-archiver`

- 1. Confirms the project directory and the slug. The vault folder does not exist, so it runs `vault_init.py create-project` first.
- 2. **Scans** with `gt_ingest.py <dir> --json` and gets a list of candidates, each with a `suggested_dest`.
- 3. **Presents by group**, asking before doing anything:

GROUP	WHAT IT FOUND	WOULD GO TO
decisions	4 CLAUDE.md rules ("never run migrations on prod without a backup")	<code>decisions.md</code>
research	6 memory files of gotchas	<code>research.md</code>
design	the README architecture section	<code>design.md</code>
knowledge	a note on the shared database's pooling limits	<code>Sources/</code> then <code>Knowledge/</code>
global_memory	the internal package registry URL	<code>global-memory/</code>
ideas	"we should build a receipt OCR service"	new project scaffold, or skip
skip	<code>user.md</code> personal prefs	excluded silently

- 4. **Asks for an explicit yes** before writing: *This will COPY files. Nothing is deleted from their current location.*
- 5. **Executes.** Decisions, research and design append as dated sections. The knowledge item is stored **immutably first** as `Sources/2026-09-05 Database Pooling Limits.md` with `title`, `local_path`, `fetches`, `supersedes: []` frontmatter, then synthesised into a `status: seed` Knowledge page citing it, and indexed. The idea becomes `Projects/receipt-ocr/idea.md` if you say scaffold.
- 6. Updates `Projects/invoice-archiver/memory/MEMORY.md` to point at what moved.
- 7. Logs `[ingest]`.
- 8. **Populates `source.md`** from the evidence it just read. The deploy scripts name three hosts through a gateway: topology `bastion-jump`. Host aliases resolved against `~/ssh/config`. A file the notes say "had to be fixed separately on each box" is recorded as `static` and unmanaged. Anything inferred is marked `TODO - verify` and listed in the summary.
- 9. Offers to wire the Golden Thread section into the repo's `CLAUDE.md`.

WHAT THIS BOUGHT YOU
A month of learned gotchas is in the vault in the right files, the originals are untouched, and the topology questions only you can answer are listed while the material is fresh. The next `/gt:gt-open` reads `source.md` and knows which of three boxes is production before anyone touches code.

/gt:gt-lint after ingest

The ingest summary says to run it. Typical findings on a fresh ingest:

- `memory-unlisted` : a moved memory file the index missed. Shows the line; you say yes.
- `orphan` : the new Knowledge page has no inbound link besides the index. It proposes linking from a related page.
- `broken-link` : a wikilink in a memory file pointed at a note that was skipped. Options: remove the link or create the target.

Declined fixes go to `lint-declines.md` so they are not re-asked.

/gt:gt-open invoice-archiver

Now the open reads a populated `source.md` first, and because it links `[[INFRASTRUCTURE]]` for the fleet, reads that page too rather than a copied host table. The summary names topology and targets so you can correct a stale entry **before** work starts. The three `TODO` – `verify` gaps are the first thing it lists.

SCENARIO 3

Production operations on a shared vault

Situation. A live incident on the order-fulfilment platform. Live customer orders, a bastion fleet, and a second Claude session is already working the same vault. This is the scenario the Core rules were built around.

/gt:gt-open order-router

The open reads `source.md` first, follows `[[INFRASTRUCTURE]]` for which host serves which role in which environment, reads the `###` headings of a long `research.md` rather than all of it, and notes the project's `validation-rules.md` pack. It reports memory files available but not loaded, and tells you the vault has N items in `review-queue.md`.

Registration and claims, before the first write

`PROTOCOL.md` and Core rule 1: register the session and claim a file before writing it. The assistant runs:

```
gt_session.py register --task "ADR-6 retry-path staging" \  
  --files Projects/order-router/research.md log.md  
gt_session.py list
```

`list` shows the other live session and what it holds. Liveness is decided by `pid` and host, not the clock. If it holds `log.md`, this session **does not edit** `log.md`. It stages its line under `Projects/golden-thread/pending/<name>.logline` to be appended when the claim clears. The `PreToolUse` guard hook denies a Write or Edit against a claimed file regardless, naming the holder.

WHAT THIS BOUGHT YOU

In one recorded incident two sessions wrote the same `research.md` minutes apart with no branches, no merge, and no conflict marker. Nothing was lost only because one checked `git status` before committing. Now a collision is prevented rather than discovered.

Working: the runbook incubator

While staging the fix you learn: *the relay only reloads its config on SIGHUP, and a restart drops all client sockets*. That is a fact about the code, useful to anyone touching it whether or not they have ever seen this vault.

The assistant writes it to `runbook.md` **immediately, with no ceremony**. That file is the incubator: the one place stability can be observed. It is not yet in the repo's `CLAUDE.md`, because a fact still in motion is not safe to publish.

Secrets: Core rule 2

Reading `order_handler.js` you find a hard-coded credential. The assistant records its **location** and that it must be rotated. It does not read the value into the session, does not print it to redact it, does not check whether it matches anything. The task line in the README says *value deliberately not recorded anywhere in the vault*.

/gt:gt-validate before a production change

`PROTOCOL.md`: a production change requires `independently verified`. The claim is *6/6 patches apply cleanly to the current production builds*. Class: `code` (does the code do what its name claims) plus `rule-compliance` (the project's rule pack, here `R2`, the

no-double-dispatch invariant). The packet carries the pack's operational half verbatim and **never** the provenance blocks between the markers, because those name the incident figures a claim may be about, and a validator handed the answer can no longer re-derive it.

The verdict lands in `research.md` labelled. A `self-verified` from the same session is recorded as exactly that, not upgraded.

/gt:gt-work

Write-back as in Scenario 1, plus:

- The runbook entry stays in `runbook.md`.
- A stable ruling becomes an ADR. When you explained *why* the flat batch-size cap is deliberate, the assistant asked in one line whether to write it as an ADR rather than absorbing it. That rule exists because a deliberate decision was written up as a defect four times over two days before an ADR recorded the reasoning.
- The stage property is checked against reality.
- The closeout probe runs; nothing fires on an active project with open P1s.
- `gt_session.py release` deletes the session file. Absence means finished.

Weeks later: /gt:gt-runbook-lint

YOU SAY
lint runbooks

1. Finds every `runbook.md` under `Projects/`, sub-projects included. Fewer than two and it stops.
2. Reads them and **clusters** near-identical content: the same SSH-through-bastion preamble in three runbooks, the same "diff before overwriting" warning in four.
3. Classifies each cluster:

CLUSTER	TYPE	DESTINATION
"diff local vs remote before scp to prod"	process rule, all projects	<code>PROTOCOL.md</code>
"the relay reloads on SIGHUP"	repo fact, one system	that project's <code>CLAUDE.md</code>
"systemd unit names on app-1"	platform fact, this stack	<code>Knowledge/</code> page
two procedures with similar wording, different steps	coincidence	left in place

4. Shows each cluster and asks: promote or leave?
5. Routes approved ones through `/gt:gt-promote`, then replaces the duplicated lines in each runbook with a pointer: `> See: Projects/PROTOCOL.md § Working Rules`.
6. Logs `[runbook-lint] N clusters, M promoted, P false positives`.

WHAT THIS BOUGHT YOU

Repetition across runbooks is the signal that a fact is general. Four copies of "diff before overwrite" become one rule in `PROTOCOL.md` that every session reads, and the four runbooks point at it instead of drifting apart.

Graduating a repo fact to `CLAUDE.md`

The SIGHUP fact survived three sessions unchanged. That is the trigger. `/gt:gt-promote` moves it to `Projects/order-router/CLAUDE.md`, which is committed to the repo root. The content rule is absolute: **self-contained**. A reader with no vault gets full value; vault paths appear only in an optional trailing section resolved through `~/.claude/vault-config.json`. This is the only channel that costs the reader nothing, because every Claude Code session in that repo reads it automatically. Logged with `graduate`, naming the destination repo from `source.md`.

Research-heavy: farming, sources, and refresh

Situation. You need to know current rate limits and pricing across three external weather APIs before deciding which one the `weather-feed` project should use. That is a lot of page reading, and the reading itself is not the scarce resource. Context is.

/gt:gt-farm

YOU SAY
farm out the API comparison

1. Applies the four gates. Most things do not leave.

GATE	QUESTION	THIS TASK
Stateless	answerable with no vault or repo state?	yes, public docs
Self-contained	fits in a paragraph plus URLs?	yes
Checkable	verifiable without redoing the work?	yes, if it cites pages
Releasable	every input safe for a third party?	only after abstracting

- The first draft mentioned account tier and volume. Those are stripped; the question is rephrased in public terms. Releasable is default-deny, and it is checked while building the packet, not before sending.
2. **Picks the vendor** from the vendor table in the farm project's `design.md`. Broad research with citations: a research API. A second opinion where being wrong is expensive: a non-Claude model, which is the one thing more Claude cannot buy.
3. **Builds the packet** in plain Markdown: `### TASK`, `### INPUTS`, a mandatory `### CURRENT VOCABULARY` block, and a strict `### RETURN CONTRACT` of `FINDINGS` (claim, page-level source or `NONE`, confidence) and mandatory `GAPS`.
4. **Supplies the vocabulary from the system, never from memory.** Model ids come from a live list call, versions from the manifest, today's date from the clock. Measured on the same packet: without a vocabulary block the model searched two generations stale and invented a rate-limit table; with it, the model scoped its queries and put the unknown in `GAPS`. The fix is prompt-side and free.
5. **Chooses plain mode**, because research mode will not honour the return contract. Depth is a rung-0, human-reads-it capability only.
6. Saves the packet to `Projects/external-research/packets/2026-09-05-api-limits.md`.
7. **Transport:** copies from the file, never from terminal text, because rendered long lines are silently truncated:

```
sed -n '/^### TASK/, $p' <packet-file> | pbcopy
```

- and tells you where to paste it. You are the transport at rung 0, which keeps it within the vendor's terms.
8. **Ingests the return.** Everything enters `unverified`. A finding becomes `self-verified` only by opening its `source:` URL and confirming the statement is on that page. `source: NONE` is a legal answer and better than an invented one. A claim that merely *follows from* a source is marked `NONE`. A return whose findings are all `NONE` is an opinion, filed as such. Results go next to the packet.

WHAT THIS BOUGHT YOU

Twenty pages of raw material never entered this context. What came back is a checkable list, and the check is a mechanical open-the-URL step rather than trust in how certain the tool sounded.

`/gt:gt-ingest` for the source that matters

One vendor's rate-limit page is the deciding fact. You say: *ingest that page*. Ingest stores it immutably as `Sources/2026-09-05 Vendor X Rate Limits.md` with `url` and `fetched`, then writes the Knowledge page citing it. Knowledge pages are summaries; Sources are ground truth. When a number matters, the source is one hop away.

`/gt:gt-validate` on the decision

Before the choice goes into `decisions.md` as an ADR, the claim *Vendor X's free tier covers our daily pull volume* is validated `empirical` against the Source file and the project's measured pull count. `decisions.md` requires `independently verified`; the ADR is written with that label.

Three months later: `/gt:gt-refresh`

YOU SAY

are my sources still current?

1. Lists every `Sources/` file with a `url:` or `local_path:` and asks which to check. Default: all remote ones. It fetches only what is in scope.
2. Fetches each and compares to the stored copy. Unchanged: noted, moves on.
3. **Changed: supersedes, never edits.** Stores the new version as `Sources/2026-12-05 Vendor X Rate Limits.md` with `supersedes: ["Sources/2026-09-05 Vendor X Rate Limits.md"]`. The old file stays on disk byte-for-byte as the historical record.
4. Greps `Knowledge/` for pages citing the old file. For each, shows what changed, proposes content updates, repoints `sources:`, sets `status: growing` if content was revised. **You approve each page update** before it is written.
5. Logs `[refresh]` with counts and one `[supersede]` line per source.

If a citing page is missed, the next `/gt:gt-lint` catches it as `superseded-cited`, the only finding that may justify `status: stale`.

WHAT THIS BOUGHT YOU

The decision in ADR-2 was made against a dated source. When that source changes, you find out, the page that summarised it is updated with your approval, and the original evidence is never overwritten. A page marked `stale` means "probably wrong," never "old," so the flag stays trustworthy.

SCENARIO 5

The interrupted week: inbox, review, and "what's next?"

Situation. Over a week of short sessions on four projects you jotted six thoughts into `INBOX.md` from wherever you were. Now you have a free afternoon and no idea where to start.

/gt:gt-review

YOU SAY
review my inbox

- 1. Reads `INBOX.md`. Every unchecked `- [ ]` line is an uncaptured item by definition. A `[project:: slug]` field is your hint; it is honoured unless clearly wrong.
- 2. Reads daily notes only if the vault has a `daily_notes_path` or a `Daily Notes/` folder. Otherwise it skips silently and does not suggest creating one.
- 3. Cross-references `Projects/README.md` to avoid duplicates.
- 4. Presents each item with a classification:

INBOX LINE	LOOKS LIKE
device-auth should share the token refresh pattern	task under <code>doorbell-camera</code>
"receipt OCR service"	new standalone project
"check whether the firewall blocks mDNS across VLANs"	task under <code>home-network</code>
"chat webhooks should live in a file not the source"	already exists as a task, skip

- 5. Asks which to promote. For a new project it asks two or three targeted questions (goal in one sentence, domain and tags, standalone or under a parent) and then runs the `/gt:gt-create` mechanics. For a task it writes under the target project's `## Tasks` with `[since:: today]` and the priority you gave. An **idea gets `p:: 3` and no due date**, because a due date on an idea makes the deadline rule rank it above real work.
- 6. Checks off each inbox line with a pointer so it reads as history:

```
- [x] device-auth should share the token refresh pattern → [[doorbell-camera]]
```

Never deleted. It registers the session and claims `INBOX.md` first, like any shared file.

- 7. Reports counts, then **regenerates the rollup**:

```
python3 Projects/golden-thread/tools/gt_tasks.py
```

"What's next?" is answered by `TASKS.md`

YOU SAY
what's next?

The assistant re-runs `gt_tasks.py` first, because project priority is computed against the clock, not stored, then reads `TASKS.md`. Its sections, top to bottom:

- 1. **Inbox:** unchecked `INBOX.md` lines. Now empty.
- 2. **Review:** projects `gt_closeout.py` thinks may be finished, with reasons. Each row is a question, answered with `gt_closeout.py` answer `<slug> yes|no|later`.
- 3. **Waiting on you:** `waiting:: user` tasks ranked `PP<effective>-P<task>`.
- 4. Then the agent's list, external, parked, and project standing.

The ranking is composed from two stored values and four computed rules:

STORED	WHERE
<code>pp</code> (project priority, 0 to 3)	project <code>README.md</code> frontmatter
<code>p</code> (task priority, 1 to 3, 7+ shelved)	the task line

COMPUTED AT THE MOMENT OF ASKING	TRIGGER
Time window	<code>pp_escalate</code> window is open right now
Stale P1	a <code>p:: 1</code> older than 7 days by <code>since</code>
Deadline	any task <code>due</code> within 3 days
Blocking	a task carries <code>blocks:: <slug></code>

None of the four applies to a shelved task or to a `complete` or `archived` project.

WHAT THIS BOUGHT YOU

The vault's demonstrated failure is not misprioritisation. It is decisions correctly made and then silently never executed: records flagged and left unactioned for two weeks, a migration scheduled twice and never run. Ranking cannot catch those. Ageing can. The stale-P1 rule is the one that matters most, and it fires without anyone remembering to look.

SCENARIO 6

The other endings: rename, merge, archive, "later"

Closing with `stage: complete` is one ending. Projects also get redefined, combined and retired. All are supported operations, because a hand-done rename leaves links pointing at nothing. Nothing is ever deleted.

Rename

The `event-relay` project turned out to be about the whole event-to-consumer chain.

YOU SAY

rename it to event-pipeline

```
vault_init.py rename-project --vault <v> --from event-relay --to event-pipeline
```

It moves the folder and re-points every wikilink it can find. `/gt:gt-lint` afterwards reports `project-missing` for any reference it could not see (Obsidian fails these silently), and you fix those by hand or leave a tombstone at the old slug.

Merge

A rehearsal project `talk-rehearsal` belongs inside `conference-talk`.

YOU SAY

merge talk-rehearsal into conference-talk

```
vault_init.py merge-project --vault <v> --from talk-rehearsal --into conference-talk
```

CONTENT	HANDLING
<code>memory/*.md</code>	moved, filenames preserved so links resolve; clashes renamed and flagged
<code>idea.md</code>	immutable, preserved verbatim as <code>memory/idea_talk-rehearsal.md</code>
<code>research.md</code>	appended; dated and append-only, so interleaving is safe
<code>decisions.md</code>	appended with ADR ids renumbered, original id kept in the heading
<code>design.md</code> , <code>source.md</code>	appended under a NEEDS REVIEW banner, not merged; two architectures cannot be combined mechanically
frontmatter	destination's kept; the question parked in <code>review-queue.md</code>

The source folder is left as a **tombstone** README recording where it went, so notes written before the merge still lead somewhere. Everything needing judgement lands in `review-queue.md`; work through it and re-run `/gt:gt-lint` before calling the merge done.

Archive

YOU SAY

archive standards-sync, superseded by the Core tier

```
vault_init.py archive-project --vault <v> --slug standards-sync --reason "superseded by core-rules"
```

`archived` is the stage; `retire` is the log verb. Every note, decision and link stays intact. The escalation rules stop applying to it.

"No" and "later"

The closeout probe asks about a project that has gone three quiet weeks (`R3`).

YOU SAY

no, it's waiting on the partner to enable the API, not finished.

```
gt_closeout.py answer partner-api no "waiting on partner API enablement"
```

The task is re-tagged `[waiting:: external]` , and the record is how the thresholds learn that three quiet weeks on an externally blocked project is not a close signal for you.

Promote a rule to Core

YOU SAY

from now on, never write a claimed file. Make that a Core rule.

That is **designation**: it is Core from that moment, no test, no prior incident required. Requiring a failure first means accepting the failure. `/gt:gt-promote` treats this as a wiring job, not a move:

1. Choose enforcement honestly: `validated` if mechanically checkable (this one is: a `PreToolUse` guard), `reminder` otherwise.
2. Set frontmatter `type: core` , `level: core` , `enforcement: validated` , `promoted` .
3. Move the file to `core-rules/core_<topic>.md` , phrased as an imperative.
4. Wire the mechanism. Reminder-tier needs no script edit; the injector reads the folder at run time. Validated-tier adds a check to the hook.
5. **Verify it fires** by running the injector and confirming the rule appears.
6. Update the pointer in `global-memory/MEMORY.md` .
7. `/gt:gt-lint: core-unenforced` must not fire.
8. Log `graduate` .

For an existing item moving up from levels 1 to 5, the three-question gate applies instead: if this were *not* enforced, would it cause incorrect code, rework, or a cascade into lower rules? Any single yes qualifies. The canary rule (timestamp every message) answers no to all three and is Core regardless, because its absence is visible: it is the health check on enforcement itself.

Demotion runs in reverse, and the order matters: **unwire first**, then move the file. A hook left pointing at a rule that no longer exists is worse than either state.

SCENARIO 7

Vault maintenance day

Once a month, with no project open. Each of these is idempotent and asks before writing.

ORDER	COMMAND	WHAT IT CATCHES
1	<code>/gt:gt-lint</code>	broken links, orphans, index gaps, unlisted memory files, global-memory files over 30 lines, project slugs leaking into global scope, pages citing superseded sources, <code>stale</code> pages, unenforced Core rules, links to renamed projects
2	<code>/gt:gt-runbook-lint</code>	facts repeated across runbooks that should be one rule somewhere higher
3	<code>/gt:gt-refresh</code>	upstream sources that changed since they were fetched
4	<code>/gt:gt-promote</code> "review promotion candidates"	<code>→ promote</code> notes in research and decisions; <code>status: seed</code> pages ready to become <code>growing</code>
5	<code>gt_closeout.py</code> candidates	projects whose signals say they may be done
6	<code>gt_tasks.py</code>	regenerate the rollup after all of the above
7	<code>/gt:gt-settings</code>	confirm nothing automatic has been switched off by accident

`gt-lint`'s `memory-bloat` finding is worth a specific note: a `global-memory/` file over 30 lines is loaded in every session for every project. The fix it proposes is to move the detail to a `Knowledge/` page and leave a pointer plus three to five essential constants. `global-scope-leak` is the companion: a project slug in `global-memory/` means a project-specific fact is being paid for everywhere.

WHAT THIS BOUGHT YOU

Structural drift is caught by a script rather than noticed by accident. Each fix is shown and approved individually, and declines are recorded in `lint-declines.md` so the same false positive is not re-argued every month.

SCENARIO 8

The gt-wiki mode: a standalone knowledge base

The `gt-wiki` plugin is the same Sources-and-Knowledge machinery without the project workspace. It is for a vault whose purpose is documentation rather than execution: a team's platform wiki, a domain reference. The five commands map onto their `gt` counterparts.

GT-WIKI COMMAND	DOES	GT EQUIVALENT
<code>/gt-wiki:gt-wiki-init</code>	<code>vault_init.py wiki --vault <path> --domain "<domain>";</code> creates <code>Sources/</code> , <code>Knowledge/</code> , <code>index.md</code> , <code>log.md</code> , <code>Knowledge/_template.md</code> , records <code>vault-config.json</code> ; runs <code>wiki_lint.py</code> and expects 0 findings	<code>/gt:gt-init</code>
<code>/gt-wiki:gt-wiki</code>	always check first before exploring repos: reads <code>index.md</code> , follows links, greps as fallback, reads <code>Sources/</code> when exact numbers matter, answers with citations, logs the query, flags contradictions between pages immediately	<code>/gt:gt-query</code>
<code>/gt-wiki:gt-wiki-ingest <url file paste></code>	duplicate check against <code>index.md</code> and <code>Sources/</code> frontmatter; stores raw content immutably; discusses which pages to create or update; waits for approval; writes one concept per page; cross-links in five directions (upstream, downstream, sibling, hub, cross-domain); updates index and log	the knowledge half of <code>/gt:gt-ingest</code>
<code>/gt-wiki:gt-wiki-lint</code>	<code>wiki_lint.py</code> : broken links, orphans, missing reciprocal links, unsourced pages, superseded citations, review-due by age (a signal, never <code>stale</code>), index mismatches, status schema, unlinked mentions; declines to <code>lint-declines.md</code>	<code>/gt:gt-lint</code>
<code>/gt-wiki:gt-wiki-refresh</code>	fetch in-scope sources, supersede changed ones with a new dated file and <code>supersedes:</code> , update citing pages with approval	<code>/gt:gt-refresh</code>

A worked path: a teammate asks *how does our deploy pipeline handle rollbacks?* `/gt-wiki:gt-wiki` reads the index, finds `Deploy Pipeline.md`, follows its link to `Rollback Procedure.md`, notices the two pages disagree on whether database migrations roll back, **flags the contradiction right away** rather than waiting for lint, answers with both citations, logs the query, and offers to file the resolved answer back as a page once you settle it. A page created from that conversation still gets a lightweight `Sources/2026-09-05 Conversation - rollback migrations.md` so the page-to-source chain stays uniform.

Command reference: where each one appears and what it saves you

COMMAND	SCENARIO	THE FAILURE IT PREVENTS
/gt:gt-init	0	rules present as documents but never re-asserted
/gt:gt-settings	0, 7	tooling acting on its own without an off switch
/gt:gt-create	1, 5	inconsistent project layouts; the origin idea lost
/gt:gt-ingest	2, 4	a month of learned gotchas stranded in a repo folder
/gt:gt-open	1, 2, 3	re-explaining context every session; editing the wrong host
/gt:gt-query	1	re-deriving what the vault already holds
/gt:gt-work	1, 3	findings and decisions evaporating at session end; stage drift
/gt:gt-validate	1, 3, 4	an unverified number reaching a decision or production
/gt:gt-farm	4	spending context on raw material instead of judgement; unverifiable answers
/gt:gt-promote	1, 3, 6, 7	facts stuck at the wrong scope; Core rules stored but unwired
/gt:gt-review	5	thoughts captured in the wrong place and lost
/gt:gt-lint	0, 2, 7	structural drift noticed by accident
/gt:gt-runbook-lint	3, 7	the same fact in four runbooks, drifting apart
/gt:gt-refresh	4, 7	a decision resting on a source that changed
gt_tasks.py	5, 7	not knowing what's next; decisions made and silently never executed
gt_closeout.py	1, 5, 6, 7	finished projects outranking live work; closure assumed rather than asked
gt_session.py	3	two sessions overwriting each other in one working tree
vault_init.py rename / merge / archive	6	hand-done lifecycle changes leaving links pointing at nothing
gt-wiki family	8	a knowledge base with no execution workspace, same guarantees

The files each session touches, and in which direction

FILE	WRITTEN BY	RULE
idea.md	gt-create, gt-review	immutable after creation
research.md	gt-work, gt-validate, gt-ingest	append-only, dated; supersede inline
decisions.md	gt-work, gt-ingest, gt-promote	append-only ADRs; requires independently verified
design.md	gt-work, gt-ingest	rewritten in place; describes now
spec.md	gt-work	created when design is settled; criteria checked off, never deleted
runbook.md	gt-work	the incubator; graduates to CLAUDE.md when stable
CLAUDE.md (project)	gt-promote	self-contained; leaves the vault with the repo
source.md	gt-create, gt-ingest, gt-work	read first on open; links the fleet, never copies it
README.md frontmatter and ## Tasks	gt-work, gt-review, closeout	properties drive the views; tasks live here, nowhere else
memory/*.md	gt-work	updated in place; indexed in MEMORY.md ; loaded on demand
Knowledge/*.md	gt-promote, gt-ingest, gt-refresh	frontmatter schema; cites Sources/
Sources/*.md	gt-ingest, gt-refresh	immutable; superseded, never edited
global-memory/*.md	gt-promote	under 30 lines; zero project slugs
core-rules/*.md	gt-promote	rules, not facts; wired and verified to fire
INBOX.md	any session	one line per thought; checked off with → [[slug]]
TASKS.md	gt_tasks.py	generated; never hand-edited
log.md	every command	append-only; closed verb vocabulary
review-queue.md	merge, lint	decisions only you can make
lint-declines.md	lint	suppressed findings with rationale
closeout-signals.jsonl	gt_closeout.py	every ask and answer, for tuning
sessions/ , pending/	gt_session.py	claims and staged edits for the concurrent case